Report 3

Exercise 2 — False Sharing

1) Comparison of the two source files

The first version allocates one integer per thread:

```
int*volatilesum= (int*)calloc(MAX_NUM_THREADS,sizeof(int));
...
sum[tid]++;
```

The second version allocates a larger array :

```
#define FACTOR 16
int*volatilesum= (int*)calloc(MAX_NUM_THREADS*FACTOR,sizeof(i
nt));
...
sum[tid*FACTOR]++;
```

on the first version, the array elements are contiguous, so several thread counters may lie in the **same cache line**.
When different threads running on different cores repeatedly update these nearby values, the cache line must constantly move between cores. This causes **false sharing**, which increases cache-coherence overhead and slows the program down.

In the second version, the padding with `FACTOR` separates the counters so that each thread is more likely to write to a different cache line. This reduces false sharing and usually improves performance.

Both versions produce the same correct result, but the padded version is expected to run faster. The reason is that it reduces false sharing by spacing the thread-local counters farther apart in memory.

2) Execution time and explanation

Execution time output :

```
===== SAME PROCESSOR =====
Total sum: 600000000
Time taken: 0.311493 seconds
Total sum: 600000000
Time taken: 0.206158 seconds
```

It shows that `false_sharing_2` is faster than `false_sharing` because it avoids **false sharing**.

In `false_sharing`, the thread counters are stored next to each other in memory, so several threads may write to values that lie in the same cache line. Even though the threads are updating different array elements, the cache line keeps being invalidated and moved between cores, which creates extra overhead and slows the program down.

In `false_sharing_2`, the counters are padded and placed farther apart, so each thread writes to a different cache line. This reduces cache-coherence traffic, which is why the execution time is lower.

3) cache behavior of the two programs :

false_sharing :

```
Performance counter stats for './false_sharing 100000000':

          2,092.37 msec task-clock:u              #    4.152 CPUs utilized
                 0      context-switches:u        #    0.000 /sec
                 0      cpu-migrations:u          #    0.000 /sec
                80      page-faults:u             #   38.234 /sec
     6,116,257,895      cycles:u                  #    2.923 GHz                      (83.32%)
     5,002,339,876      stalled-cycles-frontend:u #   81.79% frontend cycles idle     (83.27%)
       684,634,527      stalled-cycles-backend:u  #   11.19% backend cycles idle      (66.63%)
     2,410,594,197      instructions:u            #    0.39  insn per cycle
                                                  #    2.08  stalled cycles per insn  (83.40%)
       604,501,897      branches:u                  288.907 M/sec                     (83.42%)
             4,447      branch-misses:u           #    0.00% of all branches          (83.43%)

       0.503962642 seconds time elapsed

       2.070660000 seconds user
       0.001001000 seconds sys
```

false_sharing_2 :

```
Performance counter stats for './false_sharing_2 100000000':

          1,313.82 msec task-clock:u              #    5.563 CPUs utilized
                 0      context-switches:u        #    0.000 /sec
                 0      cpu-migrations:u          #    0.000 /sec
                80      page-faults:u             #   60.891 /sec
     3,815,116,050      cycles:u                  #    2.904 GHz                      (83.52%)
     2,623,222,787      stalled-cycles-frontend:u #   68.76% frontend cycles idle     (83.37%)
       585,119,554      stalled-cycles-backend:u  #   15.34% backend cycles idle      (66.37%)
     2,416,938,795      instructions:u            #    0.63  insn per cycle
                                                  #    1.09  stalled cycles per insn  (83.19%)
       605,900,061      branches:u                  461.173 M/sec                     (83.32%)
             8,170      branch-misses:u           #    0.00% of all branches          (83.48%)

       0.236191687 seconds time elapsed

       1.297850000 seconds user
       0.003949000 seconds sys
```

based on the results , we observe :

- `false_sharing` takes about **0.504 s**, while `false_sharing_2` takes about **0.236 s**
- `false_sharing` uses about **6.12 billion cycles**, while `false_sharing_2` uses about **3.82 billion cycles**
- `false_sharing` has lower **IPC** (**0.39**) than `false_sharing_2` (**0.63**)

- `false_sharing` has more **stalled cycles per instruction** (**2.08**) than `false_sharing_2` (**1.09**)

This shows that `false_sharing_2` runs more efficiently. It finishes faster, needs fewer cycles, executes more instructions per cycle, and spends less time stalled. The reason is that `false_sharing` suffers from false sharing, while `false_sharing_2` reduces that problem by padding the per-thread counters.

---

4) False sharing

Performance counter stats for './false_sharing 100000000' LLC:

```
            803      LLC-load-misses:u
            129      LLC-store-misses:u

    0.316603158 seconds time elapsed

    1.786541000 seconds user
    0.002978000 seconds sys
```

Performance counter stats for './false_sharing_2 100000000' LLC:

```
            850      LLC-load-misses:u
             97      LLC-store-misses:u

    0.211071447 seconds time elapsed

    1.231789000 seconds user
    0.005974000 seconds sys
```

Performance counter stats for './false_sharing 100000000' L1:

```
     16,797,911      L1-dcache-load-misses:u
      7,203,405      L1-dcache-store-misses:u

    0.315187560 seconds time elapsed
```

```
    1.789531000 seconds user
    0.000000000 seconds sys
```

Performance counter stats for './false_sharing_2 100000000' L1:

```
         10,820      L1-dcache-load-misses:u
          1,081      L1-dcache-store-misses:u

    0.209917913 seconds time elapsed

    1.236041000 seconds user
    0.001002000 seconds sys
```

Yes, there are significant differences.

For the **LLC misses**, the difference is small:

- `false_sharing`: about **803 load misses** and **129 store misses**
- `false_sharing_2`: about **850 load misses** and **97 store misses**

So at the last-level cache, both programs look somewhat similar.

For the **L1 cache misses**, the difference is huge:

- `false_sharing`: about **16.8 million L1 load misses** and **7.2 million L1 store misses**
- `false_sharing_2`: about **10.8 thousand L1 load misses** and **1.1 thousand L1 store misses**

This means that the real problem appears mainly in the **L1 data cache**.

The reason is false sharing. In `false_sharing`, threads write to nearby array elements, so several thread-local counters may lie in the same cache line. When one thread writes to that cache line, other cores may lose their local copy and need to reload it again. This causes many extra L1 cache misses. In `false_sharing_2`, the counters are padded and separated in memory, so each thread is more likely to use a different cache line. That greatly reduces cache invalidations and improves performance.

5) Execution time for 1 CPU and 2 CPUs

Based on the LCC3 architecture, the node has two CPUs (sockets), each with 12 hardware threads. To run the program on two different CPUs with 6 threads, we selected 3 threads from the first CPU (`0,1,2`) and 3 threads from the second CPU (`12,13,14`). In this way, the 6 OpenMP threads are distributed across both processors.